

DevSecOps Report

1. Environment Setup

I deployed WordPress locally using docker-compose. My docker-compose.yml defines two services:

- db: MySQL 8.0 with a dedicated user wpuser and the wordpress database.
- wordpress: official wordpress:6.0-apache image, exposed on port 8080 of the host.

Environment variables pass the DB credentials to the WordPress container. For this local lab I kept the passwords in clear text inside the compose file — this is deliberately loose given the context (ephemeral test environment); in production you would use secrets.

After docker compose up -d, I finished the WordPress install through a curl -X POST on /wp-admin/install.php?step=2. This avoids going through the browser and makes the scenario reproducible in CI.

When I wanted to run the vulnerable image (port 8080) and the hardened image (port 8081) in parallel for a before-and-after comparison, I separated the Compose projects using -p wp-vuln and -p wp-hardened. Otherwise, the two stacks share the same namespace by default and interfere with each other.

Challenges encountered

Several things didn't work on the first try:

1. **WPScan couldn't see WordPress.** At first I had launched the WPScan container on the same network as WordPress with --url http://wp-site, but since WordPress had been installed with siteurl=http://localhost:8080, it was redirecting the internal URLs and the scanner didn't recognize the CMS. I switched to --network host + --url http://localhost:8080 so that WPScan would see the exact same URL as the one configured in WordPress.
2. **WPScan API quota.** The free key gives 25 requests per day. After running scans locally and then in CI, I burned through the quota and the CI reported API limit reached. My first reflex was to think the key was wrong, but it was just the quota that hadn't reset yet. I had to wait for the reset and update the secret on the GitHub side.
3. **Compose project conflict.** When I launched the hardened compose, it overwrote the containers of the vulnerable compose because they shared the same default project name (the docker folder name). Solution: explicitly name the projects with -p.

How I solved them

- WPScan via --network host instead of a bridge network, to avoid the URL mismatch between install and scan.
- WPScan token stored in the GitHub secret WPSCAN_API_TOKEN and passed to the container via --api-token "\${{ secrets.WPSCAN_API_TOKEN }}".

- Isolated Compose projects with -p wp-vuln and -p wp-hardened → separate networks, volumes and containers.

2. Findings Overview

The first WPScan scan against WordPress 6.0.3 reported **21 vulnerabilities** in total: 20 on the core and 1 on the Akismet plugin (present by default in any WordPress install). The raw output is in scans/before/ (text and JSON format).

Key vulnerabilities

I grouped the findings by category:

Category	Count
Stored / DOM XSS	8
Reflected XSS	1
Path Traversal	2
CSRF	1
SSRF	1
Information disclosure	3
Arbitrary shortcode execution	2
Denial of Service	1
Deserialization of untrusted data	1
Arbitrary PHP upload (Admin+)	1
Akismet plugin	1

Risk assessment

In my view, the most serious issues are:

1. **Deserialization of untrusted data:** potential RCE if a plugin or theme exposes an exploitable gadget. Critical because it can lead directly to remote code execution.
2. **Unauthenticated Stored XSS:** an attacker without any account can inject persistent JavaScript that runs in the admin's browser.
3. **Admin+ PHP File Upload:** once an admin account is compromised, the attacker uploads a PHP webshell and gains RCE.
4. **Unauthenticated SSRF via DNS rebinding:** allows reaching internal resources (cloud metadata, private services).

The other issues have a lower impact because they require a low-level account, but they remain exploitable if registration is open or if an account is compromised.

How attackers could exploit them

The shortest attack chain I could see combines several flaws:

1. Unauth SSRF or unauth Stored XSS to get a first entry point without needing any account.

2. Reflected XSS or Contributor+ XSS to steal an admin session (session cookie, application password).
3. Admin+ PHP File Upload → upload of a webshell → code execution on the server.

Alternatively, the deserialization flaw can give a direct RCE if an exploitable gadget is present in the installed plugins, without needing to go through any account.

3. Remediation Steps

Patching and updating

The first decision was to **upgrade the version** rather than patching CVE by CVE. WordPress provides security backports on older branches (6.0.4, 6.0.5, etc.), but jumping straight to 6.8.3 fixes the 20 core vulnerabilities in a single action and also benefits from non-security improvements.

Concretely, I changed the FROM line in my Dockerfile:

FROM wordpress:6.8-apache

Hardening measures

I applied several hardening measures.

PHP (docker/hardening/php-hardening.ini):

- `expose_php = Off`: removes the X-Powered-By: PHP/... header.
- `display_errors = Off`: no info leak in error pages.
- `allow_url_fopen = Off`: blocks SSRF via `file_get_contents("http://...")`.
- `allow_url_include = Off`: blocks LFI/RFI via `include`.

Apache (docker/hardening/security.conf):

- `ServerTokens Prod + ServerSignature Off`: header `Server: Apache` instead of `Apache/2.4.65 (Debian)`. This removes version hints from attackers.
- `<Files xmlrpc.php> Require all denied`: XML-RPC blocking, which is a classic vector for brute force (`wp.getUsersBlogs`) and DDoS amplification (`pingback.ping`).
- `Headers X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Referrer-Policy: strict-origin-when-cross-origin`.
- `Header unset X-Powered-By` as an extra safeguard.

I had to name my config file `zzz-hardening.conf` because otherwise the default Debian config (`security.conf`) was loaded after mine and overrode my values (Apache loads config files in alphabetical order, and `- < .` in ASCII). I lost a bit of time on this before figuring it out.

Container:

- Apache listens on port **8080** instead of 80, which lets me run as USER `www-data` (non-root). Ports `< 1024` require root to `bind()` on Linux, so 8080 cleanly solves the problem.
- `chown -R www-data:www-data` on `/var/www/html` and `/usr/src/wordpress` before USER, so the entrypoint can write without hitting permission denied.

Attack surface reduction:

- Removal of unused default themes (`twentytwenty`, `twentytwentyone`).
- Removal of Akismet (flagged as vulnerable by WPScan) and Hello Dolly (trivial plugin, just extra code).

- `apt-get purge --auto-remove` + cleanup of `/var/lib/apt/lists/*`.

Admin lockdown:

- `define('DISALLOW_FILE_EDIT', true)` injected via the `WORDPRESS_CONFIG_EXTRA` environment variable. If an attacker compromises an admin account, they can no longer modify theme/plugin files directly from the WordPress interface.

Before/after scan evidence

I re-scanned the hardened image with exactly the same WPScan command as the vulnerable image. The artifacts are in `scans/after/`.

Measure	Before	After
WordPress version	6.0.3	6.8.3
Process user	root	www-data (uid 33)
Apache port	80	8080
xmlrpc.php	accessible (HTTP 200)	blocked (HTTP 403)
Server header	Apache/2.4.54 (Debian)	Apache
X-Powered-By	PHP/7.4.32	(absent)
X-Content-Type-Options	absent	nosniff
X-Frame-Options	absent	SAMEORIGIN
Referrer-Policy	absent	strict-origin-when-cross-origin
Akismet plugin	present (vulnerable)	removed
Admin file edit	enabled	<code>DISALLOW_FILE_EDIT</code>
WPScan	21	0
vulnerabilities		

→ 21 vulnerabilities eliminated, verified by automated scan.

4. Fixed Image Build

- **GitHub repo:** <https://github.com/alexandreTimal/FinalProjectDevops>
- **Docker Hub image:** <https://hub.docker.com/r/r0wee/wordpress-hardened> (public, anonymous pull: `docker pull r0wee/wordpress-hardened:latest`)
- **GHCR image:** `ghcr.io/alexandretimal/wordpress-hardened:latest`

The build and push are handled by the workflow `.github/workflows/build-push.yml`, which is triggered automatically when the `Dockerfile` or the `hardening/` folder changes, and which pushes simultaneously to GHCR and Docker Hub.

I had to force the GHCR name to lowercase (`${OWNER_LC,,}`) because my GitHub username has an uppercase letter and GHCR rejects non-lowercase tags.

5. Tooling Justification

Docker. Full reproducibility. The compose that runs locally is exactly the one GitHub Actions runs. No “works on my machine” possible.

Official WordPress image. Maintained by the Docker teams, regular updates, sensible defaults out of the box.

WPScan. It's the reference scanner for WordPress. It knows the WPVulnDB database (CVEs for WordPress core + plugins + themes), it can enumerate users, versions, plugins, and it works in black-box mode (HTTP only). Its JSON output is easy to parse for automated analysis. For scanning WordPress specifically, nothing else is as complete.

GitHub Actions. Integrated with the repo, free for public projects, ephemeral runners that start from a clean state on each run. The native secrets (WPSCAN_API_TOKEN, DOCKERHUB_*) are managed cleanly without any external solution. Automatic triggering on push enables shift-left.

GitHub Container Registry. Directly tied to the repo, free authentication through the native GITHUB_TOKEN (no external secret to manage just for GHCR), permissions inherited from the repo.

Docker Hub. Industry standard, widest possible distribution. Explicitly required by the assignment. It's also a good real-world test of the full chain: if the Docker Hub push succeeds in CI, it means the secrets are properly configured.

6. DevSecOps Strategy

How my workflow demonstrates shift-left security

Shift-left means moving security checks as early as possible in the development cycle, rather than at the end (audit before production, or worse, after an incident). In this lab, I implemented this through three mechanisms.

1. Automatic scan on every push. The scan.yml workflow is triggered on push: branches: [main]. Any change that lands on main triggers a full WordPress deployment + a WPScan scan + an upload of the results as an artifact. If a regression introduces a flaw (for example reverting to an older version), I see it immediately in the Actions tab. Detection is no longer a one-off human decision, it's enforced by CI.

2. Image build in CI rather than locally. The build-push.yml workflow builds the hardened image on a clean GitHub runner, using only the contents of the repo. No local artifact enters the delivered image. This guarantees that the public image on GHCR and Docker Hub is reproducible by anyone from the committed Dockerfile. No manual tinkering that would slip under the radar.

3. Closed loop: remediation → re-scan. A fixed vulnerability is only validated if the next scan proves it. This is what scans/before/ (21 vulns) and scans/after/ (0 vuln) demonstrate: remediation is measurable, dated and non-bypassable. The before/after table in the previous section is the quantified proof that the pipeline works.

Acknowledged limitations

A few points where my workflow isn't complete compared to a real DevSecOps pipeline:

- No image scanning (such as Trivy or Gype) — only application-level WordPress scanning. In production, I would add it to cover CVEs in system packages.

- No image signing (cosign) — would provide a cryptographic provenance attestation.
- DB passwords in clear text in the compose — acceptable for a local test, not for production.

These points are out of scope for the assignment, but they're what I would add to a real CI/CD chain.

Conclusion

The lab pushed me to set up a complete DevSecOps chain, not just to run tools in isolation. The scan part was relatively straightforward — WPScan does the job. What was more formative was understanding how to automate things cleanly: avoiding the set -e pitfalls, understanding that WPScan returns exit code 5 when it finds vulnerabilities (and that this is normal behavior, not a bug), managing the API quota, isolating the Compose stacks, and so on.

In the end, the pipeline shows something verifiable: 21 vulns detected on the vulnerable version, 0 on the hardened version, and all of it reproducible by anyone who clones the repo and configures the secrets. That's what shift-left is meant to deliver: turning security from a one-off audit into a continuous property of the code.